

Linear Search in Python Programming

Linear search is a simple search algorithm that checks each element in a list sequentially until the desired element is found or the entire list is searched. If the element is found, the index of that element is returned; otherwise, the algorithm returns a message that the element is not present.

Steps of Linear Search Algorithm:

1. **Start with the first element** of the list.
2. **Compare the target value** (the value you're looking for) with the current element in the list.
3. If the target value matches the current element, **return the index** of that element.
4. If the target value does not match, **move to the next element** and repeat step 2.
5. If all elements in the list have been checked and none match the target value, **return -1** (indicating that the element was not found).

Example Walkthrough

Let's perform a linear search on a sample dataset:

Sample Data (List):

[34, 12, 56, 89, 7, 23, 67]

Target Value (Element to find):23

Step-by-Step Operations:

1. **Initial List:** [34, 12, 56, 89, 7, 23, 67]
 - We are looking for the value 23.
2. **Start with the first element** (index 0):
 - Current element = 34
 - Is 34 equal to 23? No.
 - Move to the next element.
3. **Second element** (index 1):
 - Current element = 12
 - Is 12 equal to 23? No.
 - Move to the next element.
4. **Third element** (index 2):
 - Current element = 56

- Is 56 equal to 23? No.
- Move to the next element.
- 5. **Fourth element** (index 3):
 - Current element = 89
 - Is 89 equal to 23? No.
 - Move to the next element.
- 6. **Fifth element** (index 4):
 - Current element = 7
 - Is 7 equal to 23? No.
 - Move to the next element.
- 7. **Sixth element** (index 5):
 - Current element = 23
 - Is 23 equal to 23? Yes.
 - **Found:** Target value is at index 5.
- 8. The search stops and returns the **index 5**, where the value 23 was found.

If the Element Wasn't Found:

If the target value (for example, 100) was not present in the list, the search would continue until the end of the list, and since no match would be found, the algorithm would return -1.

Example of Not Found:

- If the target value is 100:
 - The algorithm goes through all the elements (34, 12, 56, 89, 7, 23, 67) and doesn't find a match.
 - Finally, it returns -1, indicating that 100 is not in the list.

Time Complexity of Linear Search:

- **Best Case:** $O(1)$ (if the element is found at the first position)
- **Worst Case:** $O(n)$ (if the element is at the last position or not present at all)

Where n is the number of elements in the list.

Linear search is a straightforward approach that checks each element in the list one by one. It works well for small datasets but is inefficient for large lists as it has to check each element.

Binary Search in Python Programming

Binary search is a more efficient search algorithm than linear search, but it only works on **sorted** lists. It repeatedly divides the search interval in half. If the value of the search key is less than the item in the middle of the interval, the search continues in the lower half. Otherwise, it continues in the upper half. This process repeats until the value is found or the interval is empty.

Steps of Binary Search Algorithm:

1. **Sort the list** (if not already sorted).
2. **Find the middle element** of the list.
3. **Compare** the middle element with the target.
4. If the target is equal to the middle element, **return the index** of the middle element.
5. If the target is less than the middle element, repeat the search on the left half of the list.
6. If the target is greater than the middle element, repeat the search on the right half of the list.
7. Continue this process until the target is found or the list cannot be divided further.

Example Walkthrough

Let's perform a **binary search** on the sorted version of the sample data:

Sample Data (List):

[34, 12, 56, 89, 7, 23, 67]

Target Value (Element to find):23

Step-by-Step Operations:

1. **Initial List:** The list is unsorted, so we need to sort it first:

Sorted List: [7, 12, 23, 34, 56, 67, 89]

Target Value: 23

Step 1: Initialize

- **Sorted List:** [7, 12, 23, 34, 56, 67, 89]
- **Target:** 23
- **Low:** 0 (index of the first element)
- **High:** 6 (index of the last element)
- **Middle index:** $\text{low} + (\text{high} - \text{low}) // 2 = 0 + (6 - 0) // 2 = 3$
- **Middle element:** 34

- **Comparison:** Is 34 equal to 23? No.

Since 23 is less than 34, we now focus on the **left half** of the list (from index 0 to 2).

Step 2: Narrow Down to Left Half

- **New Low:** 0
- **New High:** 2
- **Middle index:** $0 + (2 - 0) // 2 = 1$
- **Middle element:** 12
- **Comparison:** Is 12 equal to 23? No.

Since 23 is greater than 12, we now focus on the **right half** of the list (from index 2 to 2).

Step 3: Narrow Down to Right Half

- **New Low:** 2
- **New High:** 2
- **Middle index:** $2 + (2 - 2) // 2 = 2$
- **Middle element:** 23
- **Comparison:** Is 23 equal to 23? Yes.

Since we found the target, the **search ends here**.

- **Target found at index 2.**

Final Result:

The target value 23 is found at index 2.

If the Element Wasn't Found:

If the target value was something that does not exist in the list (e.g., 100), the search would proceed as follows:

1. **Sorted List:** [7, 12, 23, 34, 56, 67, 89]
2. **Target Value:** 100
3. **First comparison:** Middle element is 34. Since 100 is greater than 34, we look in the right half (index 4 to 6).

4. **Second comparison:** Middle element is 56. Since 100 is greater than 56, we look in the right half (index 6 to 6).
5. **Third comparison:** Middle element is 89. Since 100 is greater than 89, there are no more elements to check.
6. As there are no more elements to search, the algorithm returns -1, indicating that 100 is not in the list.

Time Complexity of Binary Search:

- **Best Case:** $O(1)$ (if the middle element is the target).
- **Worst Case:** $O(\log n)$ (each step halves the search space).

Where n is the number of elements in the list.

Summary of Operations:

1. **Sort** the list if necessary.
2. **Find the middle element** of the list.
3. **Compare** the middle element with the target.
4. If the target is found, **return the index**.
5. If the target is smaller than the middle element, **search the left half**.
6. If the target is larger than the middle element, **search the right half**.
7. Continue until the target is found or the list is exhausted

When performing **binary search** on a list with an **even number of elements**, the algorithm works exactly the same way as it does with an odd number of elements. The only difference is how the **middle element** is determined when the number of elements is even. Let's explore this in detail:

Key Points for Even-Numbered Lists:

1. When there is an **even number of elements** in the list, the **middle element** is determined by calculating the index using the formula:

$$\text{middle index} = \left\lfloor \frac{\text{low} + \text{high}}{2} \right\rfloor$$

$$\text{index} = \left\lfloor \frac{\text{low} + \text{high}}{2} \right\rfloor$$

This formula results in the **lower middle index** (i.e., it rounds down to the lower of the two middle indices).

2. After finding the middle element, you perform the same comparisons to decide whether to search the **left half** or the **right half** of the list, based on the target value.

Example Walkthrough: Even Number of Elements

Let's use a **sorted list with an even number of elements** and perform a binary search.

Sample Data (Sorted List with Even Number of Elements):

[7, 12, 23, 34, 56, 67, 89, 100]

Target Value: 56

Step-by-Step Operations:

1. **Initial List:** [7, 12, 23, 34, 56, 67, 89, 100]
 - **Target:** 56
 - **Low:** 0 (index of the first element)
 - **High:** 7 (index of the last element)
2. **Find the middle index:**
 - Formula: $\text{middle} = (\text{low} + \text{high}) // 2$
 - $\text{middle} = (0 + 7) // 2 = 3$
 - The middle element is at index 3: **34**.
3. **Comparison with Target:**
 - Is 34 equal to 56? No.
 - Since 56 is greater than 34, we now search in the **right half** (from index 4 to 7).

New Search Interval: [56, 67, 89, 100]

- **Low:** 4
- **High:** 7

Find the middle index:

- Formula: $\text{middle} = (\text{low} + \text{high}) // 2$

- $\text{middle} = (4 + 7) // 2 = 5$
- The middle element is at index 5: **67**.

Comparison with Target:

- Is 67 equal to 56? No.
- Since 56 is less than 67, we now search in the **left half** (from index 4 to 4).

New Search Interval: [56]

- **Low:** 4
- **High:** 4

Find the middle index:

- Formula: $\text{middle} = (\text{low} + \text{high}) // 2$
- $\text{middle} = (4 + 4) // 2 = 4$
- The middle element is at index 4: **56**.

Comparison with Target:

- Is 56 equal to 56? Yes.

The target 56 is found at index **4**.

If the Element Wasn't Found:

If the target value does not exist in the list (e.g., 105), the binary search would continue narrowing the search range until the interval is empty.

For example, if the target is 105:

- **Initial List:** [7, 12, 23, 34, 56, 67, 89, 100]
- **First middle element:** 34
 - Since 105 is greater than 34, search in the right half.
- **Second middle element:** 67
 - Since 105 is greater than 67, search in the right half.
- **Third middle element:** 100
 - Since 105 is greater than 100, search in the right half.

- The right half is empty, so the algorithm would return -1, indicating that 105 is not found.

Conclusion for Even-Numbered Lists:

- **Middle Element Calculation:** For an even number of elements, the algorithm chooses the **lower middle element**.
- The algorithm continues to divide the list in half and compares the target with the middle element until the target is found or the search interval is empty.
- Binary search works just as efficiently with even-numbered lists as with odd-numbered lists. The key difference is the choice of the middle element when the list has an even number of items.

Selection Sort is a simple comparison-based sorting algorithm. It works by repeatedly selecting the smallest (or largest, depending on the sorting order) element from the unsorted part of the list and swapping it with the first unsorted element. This process continues until the entire list is sorted.

Steps of the Selection Sort Algorithm:

1. **Start from the first element** of the list.
2. **Find the minimum element** from the unsorted part of the list (from the current position to the end of the list).
3. **Swap** the minimum element with the first unsorted element.
4. Move the boundary between the sorted and unsorted part one position to the right.
5. Repeat the process for the remaining unsorted part until the entire list is sorted.

Time Complexity:

- **Best Case:** $O(n^2)$
- **Average Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$
- **Space Complexity:** $O(1)$ (in-place sorting)

Example Walkthrough of Selection Sort:

Let's walk through the **Selection Sort** algorithm with an example.

Sample Data (Unsorted List):

[64, 25, 12, 22, 11]

We will sort the list in **ascending order** using the **Selection Sort** algorithm.

Step-by-Step Operations:

Iteration 1:

- **Initial List:** [64, 25, 12, 22, 11]

- The unsorted part of the list is: `[64, 25, 12, 22, 11]` (the entire list initially).
- **Find the minimum element** in this part:
 - `64, 25, 12, 22, 11` – The minimum element is `11`.
- **Swap** the minimum element `11` with the first element `64`.
- **List after swap:** `[11, 25, 12, 22, 64]`
- Now, the first element `11` is in its correct position, so the sorted part is: `[11]`.

Iteration 2:

- **Unsorted Part:** `[25, 12, 22, 64]`
- **Find the minimum element** in this part:
 - `25, 12, 22, 64` – The minimum element is `12`.
- **Swap** the minimum element `12` with the first element `25`.
- **List after swap:** `[11, 12, 25, 22, 64]`
- Now, the sorted part is: `[11, 12]`.

Iteration 3:

- **Unsorted Part:** `[25, 22, 64]`
- **Find the minimum element** in this part:
 - `25, 22, 64` – The minimum element is `22`.
- **Swap** the minimum element `22` with the first element `25`.
- **List after swap:** `[11, 12, 22, 25, 64]`
- Now, the sorted part is: `[11, 12, 22]`.

Iteration 4:

- **Unsorted Part:** `[25, 64]`
- **Find the minimum element** in this part:
 - `25, 64` – The minimum element is `25` (it's already in place, no swap needed).
- **List after no swap:** `[11, 12, 22, 25, 64]`
- Now, the sorted part is: `[11, 12, 22, 25]`.

Iteration 5:

- **Unsorted Part:** `[64]`
- There's only one element left, so it is already in its correct place.

- **Final Sorted List:** [11, 12, 22, 25, 64]

Final Result: The list is now sorted in ascending order:

[11, 12, 22, 25, 64]

Key Characteristics of Selection Sort:

- **In-Place Sorting:** Selection sort does not require extra space; it works by swapping elements within the list.
- **Comparison-Based:** It repeatedly compares elements to find the smallest (or largest) one.
- **Inefficient for Large Lists:** Due to its $O(n^2)$ time complexity, it becomes inefficient on larger lists.

Advantages:

- Simple to implement.
- Efficient on small lists.
- Works well when memory write operations are costly, as it minimizes the number of swaps.

Disadvantages:

- Not suitable for large lists due to its $O(n^2)$ time complexity.
- It does not perform well compared to more advanced algorithms like quicksort or mergesort.

Bubble Sort is another simple sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The process continues until the list is sorted. It is called "bubble sort" because smaller elements "bubble" to the top (beginning of the list) with each pass through the list.

Steps of the Bubble Sort Algorithm:

1. **Start at the beginning** of the list.
2. **Compare adjacent elements.** If the first element is greater than the second, **swap them**.
3. **Repeat the process** for the entire list, moving to the next element until the last pair.
4. After each pass through the list, the largest element "bubbles" up to its correct position at the end of the list.
5. **Repeat the process** for the remaining unsorted portion of the list, each time reducing the range of comparison as the largest elements are sorted into their correct position.

Time Complexity:

- **Best Case:** $O(n)$ (if the list is already sorted, only one pass is needed).
- **Average Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$
- **Space Complexity:** $O(1)$ (in-place sorting)

Example Walkthrough of Bubble Sort:

Let's walk through the **Bubble Sort** algorithm with an example.

Sample Data (Unsorted List):

[64, 34, 25, 12, 22, 11]

We will sort the list in **ascending order** using the **Bubble Sort** algorithm.

Step-by-Step Operations:

Iteration 1:

- **Initial List:** [64, 34, 25, 12, 22, 11]
- **Pass 1:** Compare adjacent elements and swap if needed.
 - Compare 64 and 34: Since $64 > 34$, swap them. **New List:** [34, 64, 25, 12, 22, 11]
 - Compare 64 and 25: Since $64 > 25$, swap them. **New List:** [34, 25, 64, 12, 22, 11]
 - Compare 64 and 12: Since $64 > 12$, swap them. **New List:** [34, 25, 12, 64, 22, 11]
 - Compare 64 and 22: Since $64 > 22$, swap them. **New List:** [34, 25, 12, 22, 64, 11]
 - Compare 64 and 11: Since $64 > 11$, swap them. **New List:** [34, 25, 12, 22, 11, 64]
- After **Pass 1**, the largest element 64 has "bubbled" to its correct position at the end of the list.

Iteration 2:

- **Unsorted Part:** [34, 25, 12, 22, 11]
- **Pass 2:** Compare adjacent elements and swap if needed.
 - Compare 34 and 25: Since $34 > 25$, swap them. **New List:** [25, 34, 12, 22, 11, 64]
 - Compare 34 and 12: Since $34 > 12$, swap them. **New List:** [25, 12, 34, 22, 11, 64]
 - Compare 34 and 22: Since $34 > 22$, swap them. **New List:** [25, 12, 22, 34, 11, 64]
 - Compare 34 and 11: Since $34 > 11$, swap them. **New List:** [25, 12, 22, 11, 34, 64]
- After **Pass 2**, the second-largest element 34 has "bubbled" to its correct position at index 4.

Iteration 3:

- **Unsorted Part:** [25, 12, 22, 11]
- **Pass 3:** Compare adjacent elements and swap if needed.
 - Compare 25 and 12: Since $25 > 12$, swap them. **New List:** [12, 25, 22, 11, 34, 64]
 - Compare 25 and 22: Since $25 > 22$, swap them. **New List:** [12, 22, 25, 11, 34, 64]
 - Compare 25 and 11: Since $25 > 11$, swap them. **New List:** [12, 22, 11, 25, 34, 64]

- After **Pass 3**, the third-largest element 25 has "bubbled" to its correct position at index 3.

Iteration 4:

- **Unsorted Part:** [12, 22, 11]
- **Pass 4:** Compare adjacent elements and swap if needed.
 - Compare 12 and 22: No swap needed because $12 < 22$.
 - Compare 22 and 11: Since $22 > 11$, swap them. **New List:** [12, 11, 22, 25, 34, 64]
- After **Pass 4**, the fourth-largest element 22 has "bubbled" to its correct position at index 2.

Bubble Sort is another simple sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The process continues until the list is sorted. It is called "bubble sort" because smaller elements "bubble" to the top (beginning of the list) with each pass through the list.

Steps of the Bubble Sort Algorithm:

1. **Start at the beginning** of the list.
2. **Compare adjacent elements.** If the first element is greater than the second, **swap them**.
3. **Repeat the process** for the entire list, moving to the next element until the last pair.
4. After each pass through the list, the largest element "bubbles" up to its correct position at the end of the list.
5. **Repeat the process** for the remaining unsorted portion of the list, each time reducing the range of comparison as the largest elements are sorted into their correct position.

Time Complexity:

- **Best Case:** $O(n)$ (if the list is already sorted, only one pass is needed).
- **Average Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$
- **Space Complexity:** $O(1)$ (in-place sorting)

Example Walkthrough of Bubble Sort:

Let's walk through the **Bubble Sort** algorithm with an example.

Sample Data (Unsorted List):

csharp

Copy code

```
[64, 34, 25, 12, 22, 11]
```

We will sort the list in **ascending order** using the **Bubble Sort** algorithm.

Step-by-Step Operations:

Iteration 1:

- **Initial List:** [64, 34, 25, 12, 22, 11]
 - **Pass 1:** Compare adjacent elements and swap if needed.
 - Compare 64 and 34: Since $64 > 34$, swap them. **New List:** [34, 64, 25, 12, 22, 11]
 - Compare 64 and 25: Since $64 > 25$, swap them. **New List:** [34, 25, 64, 12, 22, 11]
 - Compare 64 and 12: Since $64 > 12$, swap them. **New List:** [34, 25, 12, 64, 22, 11]
 - Compare 64 and 22: Since $64 > 22$, swap them. **New List:** [34, 25, 12, 22, 64, 11]
 - Compare 64 and 11: Since $64 > 11$, swap them. **New List:** [34, 25, 12, 22, 11, 64]
 - After **Pass 1**, the largest element 64 has "bubbled" to its correct position at the end of the list.
-

Iteration 2:

- **Unsorted Part:** [34, 25, 12, 22, 11]

- **Pass 2:** Compare adjacent elements and swap if needed.
 - Compare 34 and 25: Since $34 > 25$, swap them. **New List:** [25, 34, 12, 22, 11, 64]
 - Compare 34 and 12: Since $34 > 12$, swap them. **New List:** [25, 12, 34, 22, 11, 64]
 - Compare 34 and 22: Since $34 > 22$, swap them. **New List:** [25, 12, 22, 34, 11, 64]
 - Compare 34 and 11: Since $34 > 11$, swap them. **New List:** [25, 12, 22, 11, 34, 64]
 - After **Pass 2**, the second-largest element 34 has "bubbled" to its correct position at index 4.
-

Iteration 3:

- **Unsorted Part:** [25, 12, 22, 11]
 - **Pass 3:** Compare adjacent elements and swap if needed.
 - Compare 25 and 12: Since $25 > 12$, swap them. **New List:** [12, 25, 22, 11, 34, 64]
 - Compare 25 and 22: Since $25 > 22$, swap them. **New List:** [12, 22, 25, 11, 34, 64]
 - Compare 25 and 11: Since $25 > 11$, swap them. **New List:** [12, 22, 11, 25, 34, 64]
 - After **Pass 3**, the third-largest element 25 has "bubbled" to its correct position at index 3.
-

Iteration 4:

- **Unsorted Part:** [12, 22, 11]
 - **Pass 4:** Compare adjacent elements and swap if needed.
 - Compare 12 and 22: No swap needed because $12 < 22$.
 - Compare 22 and 11: Since $22 > 11$, swap them. **New List:** [12, 11, 22, 25, 34, 64]
 - After **Pass 4**, the fourth-largest element 22 has "bubbled" to its correct position at index 2.
-

Iteration 5:

- **Unsorted Part:** [12, 11]
- **Pass 5:** Compare adjacent elements and swap if needed.
 - Compare 12 and 11: Since $12 > 11$, swap them. **New List:** [11, 12, 22, 25, 34, 64]
- After **Pass 5**, the list is fully sorted.

Final Sorted List:

[11, 12, 22, 25, 34, 64]

Key Characteristics of Bubble Sort:

- **In-Place Sorting:** Bubble sort does not require extra memory, as it sorts the list by swapping elements in place.
- **Simple to Implement:** It is straightforward and easy to understand, making it suitable for small datasets or educational purposes.
- **Inefficient for Large Lists:** Bubble sort is inefficient for large datasets because it has a time complexity of $O(n^2)$, which means its performance worsens quickly as the size of the list increases.

Advantages:

- **Easy to Implement:** Its simplicity makes it a great choice for beginners.
- **Stable Sort:** Bubble sort is a stable sort, meaning that it preserves the relative order of equal elements.
- **In-Place:** It doesn't require additional memory beyond the input list.

Disadvantages:

- **Slow:** Due to its $O(n^2)$ time complexity, it is very slow on large lists, and there are much more efficient algorithms like quicksort and mergesort.

Merge Sort is a highly efficient, comparison-based sorting algorithm that follows the **Divide and Conquer** strategy. It divides the list into smaller sublists, sorts those sublists, and then merges them back together to form the final sorted list. Merge Sort is widely used due to its efficient time complexity of $O(n \log n)$, even in the worst case.

Steps of the Merge Sort Algorithm:

1. **Divide**: Split the list into two halves.
2. **Conquer**: Recursively sort each half.
3. **Combine**: Merge the two sorted halves back together into a single sorted list.

Merge Sort Algorithm:

- **Divide** the list into two halves.
- **Recursively** apply merge sort to each half until you reach lists of size 1 (which are already sorted).
- **Merge** the sorted halves back together, comparing the elements of the two halves and combining them in the correct order.

Time Complexity:

- **Best Case**: $O(n \log n)$ (this is the same for all cases, because merge sort always divides the list in half).
- **Average Case**: $O(n \log n)$
- **Worst Case**: $O(n \log n)$
- **Space Complexity**: $O(n)$ (extra space is required for the temporary sublists).

Example Walkthrough of Merge Sort:

Let's walk through the **Merge Sort** algorithm with an example.

Sample Data (Unsorted List):

[64, 34, 25, 12, 22, 11]

We will sort the list in **ascending order** using **Merge Sort**

Step-by-Step Operations:

Step 1: Divide the List

The list is divided into two halves:

[64, 34, 25] and [12, 22, 11]

Step 2: Divide Each Half Recursively

- For the left half [64, 34, 25], divide it further:

[64] and [34, 25]

[64] is already sorted because it's a single element.

Now, sort [34, 25]:

- Divide [34, 25] into:

[34] and [25]

- Both [34] and [25] are single elements, so they are already sorted.

Step 3: Merge the Sorted Sublists

Now, start merging the sublists back together:

- Merge [34] and [25]:
 - Compare 34 and 25. Since 25 is smaller, the merged result is [25, 34].
- Now, merge [64] with [25, 34]:
 - Compare 64 with 25. Since 25 is smaller, the merged result is [25, 34, 64].

So, the left half [64, 34, 25] is now sorted: [25, 34, 64].

Step 4: Recursively Sort the Right Half

- For the right half [12, 22, 11], divide it further:

[12] and [22, 11]

[12] is already sorted because it's a single element.

Now, sort [22, 11]:

- Divide [22, 11] into:

[22] and [11]

- Both [22] and [11] are single elements, so they are already sorted.

Step 5: Merge the Sorted Sublists

Now, start merging the sublists back together:

- Merge [22] and [11]:
 - Compare 22 and 11. Since 11 is smaller, the merged result is [11, 22].
- Now, merge [12] with [11, 22]:
 - Compare 12 with 11. Since 11 is smaller, the merged result is [11, 12, 22].

So, the right half [12, 22, 11] is now sorted: [11, 12, 22].

Step 6: Merge the Two Sorted Halves

Now, merge the two sorted halves [25, 34, 64] and [11, 12, 22]:

- Compare 25 with 11. Since 11 is smaller, the merged result is [11].
- Compare 25 with 12. Since 12 is smaller, the merged result is [11, 12].
- Compare 25 with 22. Since 22 is smaller, the merged result is [11, 12, 22].

- Compare 25 with 34. Since 25 is smaller, the merged result is [11, 12, 22, 25].
- Compare 34 with 64. Since 34 is smaller, the merged result is [11, 12, 22, 25, 34].
- Finally, add 64 to the list: [11, 12, 22, 25, 34, 64].

Final Sorted List:

[11, 12, 22, 25, 34, 64]

Explanation of Merge Sort Process:

1. Divide the list into two halves.
2. Recursively split each half into smaller sublists until each sublist has only one element.
3. Merge the sublists back together, comparing the elements and arranging them in the correct order.
4. Repeat until the entire list is sorted.

Advantages of Merge Sort:

- Time Complexity: It has a consistent time complexity of $O(n \log n)$ in all cases (best, average, and worst), making it much more efficient than algorithms like bubble sort or selection sort.
- Stable Sort: Merge sort is stable, meaning that the relative order of equal elements is maintained.
- Works Well for Large Data: Due to its $O(n \log n)$ complexity, merge sort performs well even with large datasets.

Disadvantages of Merge Sort:

- Space Complexity: Merge sort requires additional space ($O(n)$) to store the temporary sublists during the merge process, making it less efficient in terms of memory usage compared to in-place sorting algorithms like quicksort.
- Not In-Place Sorting: Unlike bubble sort or quicksort, merge sort does not perform in-place sorting, which means it requires extra memory for the sublists during the merge phase.

Conclusion:

Merge Sort is a highly efficient sorting algorithm that works well on large datasets. Its divide-and-conquer approach ensures that it sorts in $O(n \log n)$ time, making it one of the fastest general-purpose sorting algorithms. However, it does use extra memory for the temporary arrays, which is a consideration when dealing with memory-limited systems.